

Содержание

Реферат	2
Введение	3
1. Интеллектуальное ранжирование кандидатов в ATS: предметная область и постановка задачи.....	5
2. Формализация задачи релевантности и выбор метрик качества	8
3. Архитектура и реализация экспериментального bi-encoder пайплайна.....	12
4. Экспериментальная проверка, интеграционное тестирование и анализ результатов.....	16
Заключение.....	23
Список использованных источников.....	24
Приложение А. Профилирование набора данных	26
Приложение Б. Протокол экспериментальных запусков.....	28
Приложение В. Материалы интеграционной проверки.....	30
Приложение Г. Фрагменты конфигурации и рабочих сценариев.....	31
Приложение Д. Анализ рисков внедрения и качества данных	32
Приложение Е. Методические замечания по воспроизводимости экспериментов.....	34

Реферат

Отчет содержит результаты производственной практики, выполненной в рамках разработки интеллектуального модуля ранжирования кандидатов для ATS-системы. Работа была направлена не только на получение экспериментальной модели, но и на доведение прототипа до состояния, в котором его можно воспроизводимо обучать, проверять на тестовом наборе и использовать через прикладной API.

В ходе практики рассмотрена задача сопоставления текстов вакансий и резюме. Основное внимание уделено подходу bi-encoder: вакансии и резюме кодируются независимо, после чего их соответствие оценивается по близости в векторном пространстве. Такой подход выбран как компромисс между качеством семантического анализа и вычислительной стоимостью, что особенно важно для ATS-контуров, где ранжирование должно выполняться интерактивно и масштабироваться на большой пул кандидатов.

В экспериментальной части использован тестовый набор, содержащий 521 пару «вакансия–резюме», 423 уникальные вакансии и 510 уникальных резюме. Из общего числа пар 177 размечены как релевантные. Для оценки качества применялись метрики NDCG@K, Precision@K, Recall@K и MRR. Полученные результаты показывают работоспособность выбранного подхода, однако интерпретируются с учетом структуры тестового набора: часть вакансий не имеет позитивной разметки, а число кандидатов на отдельные вакансии неодинаково.

В инженерной части реализованы воспроизводимый экспериментальный пайплайн, модуль ранжирования ATSRanker и сервисный контур на FastAPI. Проверены сценарии индексации кандидатов, ранжирования по вакансии, точечного скоринга пары «вакансия–кандидат» и передачи результата во внешний контур ATS. Дополнительно зафиксированы ограничения текущей постановки и предложены направления дальнейшего развития: расширение выборки, контроль качества разметки, сравнение с baseline-моделями, мониторинг деградации и развитие объяснимости ранжирования.

Ключевые слова: ATS, ранжирование кандидатов, резюме, вакансии, bi-encoder, rubert-tiny2, sentence-transformers, семантическое сопоставление, cosine similarity, NDCG, Precision@K, Recall@K, MRR, FastAPI, интеграционное тестирование.

Введение

Автоматизация подбора персонала является одной из практических областей, где методы обработки естественного языка дают непосредственный прикладной эффект. В крупных организациях поток резюме и откликов может быть значительным, а первичный анализ кандидатов занимает существенную часть времени рекрутера. При этом сам процесс отбора редко сводится к простому совпадению ключевых слов: один и тот же навык может быть описан разными формулировками, опыт кандидата может быть распределен по нескольким разделам резюме, а требования вакансии часто содержат как обязательные, так и желательные признаки.

ATS-системы решают задачу хранения и маршрутизации кандидатов, однако без интеллектуального ранжирования они остаются преимущественно учетным инструментом. Практически значимая система должна не только хранить карточки резюме, но и помогать пользователю быстрее находить кандидатов, наиболее соответствующих конкретной вакансии. В этом смысле интеллектуальный модуль ранжирования выступает как промежуточный слой между неструктурированными текстовыми данными и прикладным решением рекрутера.

Производственная практика была посвящена разработке и отладке такого модуля. Работа продолжает направление магистерского исследования, связанного с применением NLP-методов в задачах автоматизированного подбора персонала. В отличие от чисто исследовательского прототипа, в рамках практики акцент был сделан на воспроизводимость, проверяемость и интеграционную пригодность решения. Это означает, что модель должна быть не только обучена, но и встроена в понятный экспериментальный контур: данные должны быть подготовлены, сценарии запуска — формализованы, метрики — рассчитаны, а API — проверен в условиях, близких к реальному использованию.

Объектом работы является процесс интеллектуального отбора кандидатов в ATS-системе.

Предметом работы являются методы семантического сопоставления вакансий и резюме, архитектурные решения для ранжирования кандидатов и метрики качества, позволяющие оценивать корректность выдачи.

Цель практики — разработать и экспериментально проверить модуль ранжирования кандидатов, который способен принимать описание вакансии и набор кандидатских профилей, применять обязательные ограничения, вычислять семантическое соответствие и возвращать упорядоченный список кандидатов в формате, пригодном для интеграции с ATS.

Для достижения поставленной цели были решены следующие задачи:

1. Проанализированы особенности данных в предметной области рекрутмента и сформулированы требования к модулю ранжирования.
2. Рассмотрены подходы к сопоставлению текстов вакансий и резюме: лексические методы, cross-encoder модели, bi-encoder архитектуры и гибридные схемы.
3. Формализована функция релевантности пары «вакансия–резюме» и выбран набор метрик качества ранжирования.
4. Реализован экспериментальный пайплайн подготовки данных, обучения модели и оценки качества.
5. Разработан ранжирующий модуль, объединяющий обязательную фильтрацию и семантический скоринг.
6. Подготовлен API-контур для интеграционного использования решения.
7. Выполнена экспериментальная и интеграционная проверка, проанализированы полученные результаты и ограничения текущего решения.

Практическая значимость работы состоит в том, что полученный модуль может использоваться как основа для последующего развития интеллектуального компонента ATS. Он формирует машиночитаемый результат ранжирования, поддерживает проверяемые сценарии запуска и позволяет перейти от единичного исследовательского эксперимента к воспроизводимой инженерной процедуре. Вместе с тем результаты практики не следует трактовать как завершенное промышленное внедрение: они фиксируют работоспособный прототип и набор экспериментальных выводов, которые требуют дальнейшей проверки на расширенных данных.

1. Интеллектуальное ранжирование кандидатов в ATS: предметная область и постановка задачи

1.1. Особенности рекрутинговых данных и ограничения ATS-систем

Процесс подбора персонала включает несколько взаимосвязанных этапов: публикацию вакансии, сбор откликов, первичный скрининг, сопоставление требований с профилями кандидатов, передачу кандидатов на следующие этапы и фиксацию итогового решения. В традиционном ручном процессе значительная часть времени уходит на чтение резюме и проверку базового соответствия требованиям вакансии. При большом количестве откликов это становится узким местом: рекрутер вынужден просматривать множество однотипных карточек, при этом вероятность пропустить релевантного кандидата увеличивается.

ATS-система снижает организационные издержки: она централизует вакансии, кандидатов, статусы, историю взаимодействий и результаты отбора. Однако сама по себе централизация данных не решает задачу интеллектуального выбора. Если система не умеет оценивать степень соответствия кандидата вакансии, пользователь все равно вынужден выполнять смысловое сопоставление вручную. Поэтому в современных ATS возрастает роль модулей ранжирования и рекомендательных механизмов.

Данные в рекрутинге обладают рядом особенностей, которые затрудняют прямое применение простых правил. Во-первых, резюме и вакансии содержат неоднородные поля: часть информации представлена структурированно, часть — свободным текстом. Во-вторых, один и тот же навык может иметь разные языковые формы: например, «разработка REST API», «создание backend-сервисов» и «проектирование HTTP-интерфейсов» могут описывать пересекающиеся компетенции. В-третьих, текст вакансии часто содержит не только жесткие требования, но и контекст: описание команды, технологического стека, уровня самостоятельности, формата работы. В-четвертых, резюме кандидата может содержать релевантный опыт в разных разделах, причем не всегда в виде явного списка ключевых навыков.

Кроме языковой неоднородности, существуют и инженерные ограничения. Ранжирование должно выполняться за приемлемое время, иначе модуль будет неудобен для интерактивного интерфейса. При этом решение должно быть воспроизводимым: одинаковый запрос при одинаковой версии модели и данных должен давать сопоставимый результат. Наконец, для прикладного использования важна контролируемость: рекрутеру и владельцу

системы должно быть понятно, почему кандидат попал в топ выдачи и какие ограничения были применены до семантического скоринга.

С учетом этих условий задача ранжирования в ATS не может быть сведена только к поиску ключевых слов. Необходим подход, который учитывает смысловое сходство текстов, но при этом допускает инженерную реализацию в сервисном контуре. Именно поэтому в данной работе рассматривается многоступенчатая схема: сначала применяются обязательные фильтры, затем для оставшегося пула кандидатов рассчитывается семантическая близость.

1.2. Подходы к сопоставлению вакансий и резюме

В задачах сопоставления текстовых документов можно выделить несколько групп методов, каждая из которых имеет свои преимущества и ограничения.

Лексические методы основаны на совпадении слов и выражений. К этой группе относятся TF-IDF, BM25 и близкие им подходы. Их сильная сторона — простота, скорость и интерпретируемость. Если вакансия содержит редкий технологический термин, а резюме кандидата содержит тот же термин, лексическая модель быстро зафиксирует совпадение. Такие методы удобно использовать в качестве базовой линии и как часть гибридного поиска. Однако они слабо учитывают синонимию, контекст и различие формулировок. В рекрутинговой предметной области это критично: кандидат может обладать требуемым опытом, но описать его словами, которые не совпадают буквально с текстом вакансии.

Cross-encoder модели оценивают пару текстов совместно. Вакансия и резюме подаются на вход одной модели, которая строит представление пары и выдает оценку соответствия. Такой подход обычно обеспечивает высокое качество, поскольку модель видит оба текста одновременно и может учитывать сложные взаимодействия между фрагментами. Основной недостаток — вычислительная стоимость. Если для одной вакансии необходимо оценить тысячи кандидатов, cross-encoder должен выполнить отдельный прогон для каждой пары. Для онлайн-ранжирования в ATS это может быть слишком дорого, особенно при ограниченных ресурсах.

Bi-encoder подход строит представления вакансий и резюме независимо. Каждый текст кодируется в вектор фиксированной размерности, а затем соответствие пары оценивается через меру близости, например косинусное сходство. Важное преимущество состоит в том, что эмбединги резюме можно заранее посчитать и хранить в индексе. При появлении новой вакансии достаточно вычислить ее эмбединг и сравнить его с уже подготовленными кандидатскими представлениями. За счет этого bi-encoder хорошо подходит для задач dense retrieval и масштабируемого ранжирования.

Гибридные схемы объединяют несколько этапов. Например, на первом шаге могут применяться жесткие фильтры по обязательным требованиям, затем лексический поиск или dense retrieval формирует кандидатный пул, а на финальном шаге более дорогая модель уточняет порядок в топе. Такой подход часто оказывается наиболее практичным: он позволяет сохранить управляемость бизнес-правил и одновременно использовать семантическую информацию.

В рамках данной практики выбран bi-encoder подход в составе многоступенчатого пайплайна. Этот выбор обусловлен несколькими соображениями. Во-первых, он обеспечивает приемлемый баланс между качеством и вычислительной стоимостью. Во-вторых, он хорошо согласуется с сервисной архитектурой ATS, где часть данных можно индексировать заранее. В-третьих, такой подход проще воспроизводимо проверять: входные тексты, эмбединги, значения similarity и итоговый порядок кандидатов могут быть зафиксированы как экспериментальные артефакты.

1.3. Постановка задачи практики и критерии успешности

Задача практики сформулирована как разработка и апробация модуля ранжирования кандидатов, который принимает описание вакансии и набор профилей кандидатов, выполняет предварительную фильтрацию, рассчитывает семантическое соответствие и возвращает упорядоченный список кандидатов.

В прикладной постановке входными данными являются:

- текстовое и частично структурированное описание вакансии;
- набор кандидатских профилей, включающих текст резюме и дополнительные атрибуты;
- набор обязательных ограничений, которые должны применяться до семантического ранжирования;
- параметры выдачи, включая размер top-K и режим расчета скоринга.

Выходными данными является список кандидатов, отсортированный по убыванию релевантности. Для каждого кандидата в результате должны быть доступны идентификатор, итоговый score, служебная информация о примененных этапах обработки и телеметрия выполнения запроса. В сервисном контуре результат должен передаваться в JSON-формате, чтобы его можно было использовать в интерфейсе ATS или в downstream-сценариях.

В ходе практики были приняты следующие ограничения:

- ориентация на русскоязычные тексты вакансий и резюме;
- использование модели умеренного размера, пригодной для экспериментов при ограниченных ресурсах;

- воспроизводимость подготовки данных, обучения и оценки;
- возможность интеграционной проверки через API;
- сохранение связи между экспериментальными результатами и отчетными материалами.

Критерии успешности практики включали четыре уровня. Первый уровень — исследовательский: должна быть обоснована постановка задачи и выбран методический подход. Второй уровень — экспериментальный: должен быть реализован пайплайн обучения и оценки модели. Третий уровень — инженерный: должен быть подготовлен ранжирующий модуль и API-контур. Четвертый уровень — отчетный: результаты должны быть документированы в форме, пригодной для аттестации и дальнейшего использования в магистерской работе.

Таким образом, практика рассматривалась не как изолированный запуск модели, а как полный цикл прикладной разработки: от анализа предметной области до получения проверяемого модуля и фиксации результатов.

Выводы

1. Задача ранжирования кандидатов в ATS требует учета как семантики текстов, так и прикладных ограничений рекрутингового процесса.
2. Лексические методы полезны как базовая линия и инструмент точного поиска, но недостаточны для устойчивой обработки вариативных формулировок.
3. Cross-encoder модели потенциально дают высокое качество, однако их вычислительная стоимость затрудняет применение к большому пулу кандидатов в онлайн-сценарии.
4. Bi-encoder архитектура является рациональным компромиссом для текущего этапа: она поддерживает независимое кодирование текстов, быстрый расчет близости и воспроизводимую экспериментальную проверку.
5. Практическая постановка задачи включает не только обучение модели, но и подготовку ранжирующего API, интеграционные проверки и документирование ограничений.

2. Формализация задачи релевантности и выбор метрик качества

2.1. Представление вакансий и резюме в задаче семантического сопоставления

Для формального описания задачи введем множество вакансий V и множество кандидатов S . Каждая вакансия $v \in V$ задается набором признаков, включающих название роли, уровень позиции, ключевые требования, стек технологий, описание задач и дополнительные условия. Каждый кандидат $s \in S$ описывается резюме и сопутствующими атрибутами: опытом, навыками, локацией, уровнем, форматом занятости и другими признаками, если они доступны.

На практике исходные данные редко представлены в идеально нормализованном виде. Поэтому перед подачей в модель требуется сформировать текстовое представление вакансии и резюме. Такое представление должно быть достаточно полным, чтобы сохранить смысловую информацию, но не должно превращаться в случайный конкатенированный набор полей. В экспериментальном пайплайне использовалась идея объединения ключевых полей в единый текстовый документ, где сохраняются роль, навыки, уровень и контекстное описание.

Bi-encoder модель отображает каждый текст в векторное пространство. После этого релевантность пары «вакансия–кандидат» оценивается по близости эмбедингов. В текущей реализации использовалась косинусная близость $\text{sim}(v, c) = \text{cosine}(e_v, e_c)$.

Чем выше значение $\text{sim}(v, c)$, тем выше кандидат располагается в ранжированном списке. При этом важно подчеркнуть, что score модели не является абсолютной вероятностью найма или универсальной оценкой качества кандидата. Это именно оценка семантического соответствия между конкретной вакансией и конкретным резюме в рамках выбранного текстового представления и обученной модели.

Такое уточнение принципиально важно для корректного применения модуля. Ранжирование не должно заменять профессиональное решение рекрутера или нанимающего менеджера. Его задача — ускорить первичный анализ и помочь выделить кандидатов, которых целесообразно рассмотреть в первую очередь.

2.2. Фильтрация, скоринг и функция ранжирования

В реальном рекрутинговом процессе не все условия вакансии равнозначны. Часть признаков может быть обязательной: например, локация, формат занятости, наличие определенного допуска, минимальный уровень опыта или владение критической технологией. Если кандидат не удовлетворяет таким ограничениям, высокий семантический score не должен автоматически поднимать его в выдаче.

Поэтому в архитектуре модуля выделяются два этапа.

Первый этап — *hard filtering*. На этом шаге применяются обязательные правила, которые отсеивают кандидатов, не проходящих базовые условия вакансии. Такой подход делает поведение системы более управляемым и понятным для пользователя. Кроме того, он уменьшает размер пула, для которого требуется выполнять семантический скоринг.

Второй этап — *dense reranking*. Для кандидатов, прошедших фильтрацию, рассчитывается близость между эмбедингом вакансии и эмбедингами резюме. Далее кандидаты сортируются по убыванию значения *similarity*. Итоговая функция ранжирования может быть записана в виде:

$\text{rank}(v, C) = \text{sort_desc}(\{(c, \text{sim}(v, c)) \mid c \in C, F(v, c) = \text{true}\}),$

где $F(v, c)$ — функция обязательной фильтрации.

Такое разделение важно не только технически, но и методически. Если смешать все признаки в один общий score без явного разделения, система станет менее прозрачной: будет трудно объяснить, почему кандидат с нарушением обязательного условия оказался в верхней части выдачи. В текущем прототипе выбран более контролируемый подход: бизнес-ограничения обрабатываются до семантической модели, а модель отвечает за порядок внутри допустимого множества кандидатов.

2.3. Метрики качества и особенности их интерпретации

Для оценки ранжирования использовался набор метрик, отражающих разные стороны качества выдачи. Использование одной метрики было бы недостаточным, поскольку рекрутинговая задача сочетает несколько требований: релевантные кандидаты должны находиться высоко, топ выдачи должен быть точным, а система должна по возможности покрывать все релевантные варианты.

$\text{Precision}@K$ показывает долю релевантных кандидатов среди первых K элементов выдачи. Эта метрика особенно важна для пользовательского сценария, в котором рекрутер просматривает ограниченное число кандидатов. Если $\text{Precision}@5$ высока, значит первые пять позиций в выдаче преимущественно содержат релевантные профили.

$\text{Recall}@K$ показывает, какую долю всех релевантных кандидатов удалось найти в первых K позициях. В отличие от $\text{Precision}@K$, эта метрика оценивает полноту. Она важна в ситуациях, когда пропуск релевантного кандидата нежелателен.

$\text{NDCG}@K$ учитывает не только факт наличия релевантных кандидатов в $\text{top-}K$, но и их позиции. Релевантный кандидат на первой позиции дает больший вклад, чем тот же кандидат на пятой позиции. Для ранжирования эта метрика особенно информативна, поскольку порядок выдачи является центральным результатом работы модели.

MRR фиксирует среднюю обратную позицию первого релевантного кандидата. Эта метрика удобна для сценариев быстрого выбора, когда важно, насколько быстро пользователь увидит хотя бы один хороший вариант.

В рамках практики расчет метрик выполнялся с учетом структуры тестовой выборки. По позитивным запросам были получены высокие значения NDCG и MRR . Однако такие результаты требуют аккуратной интерпретации. Если часть вакансий имеет малый кандидатный пул или только одну явно релевантную пару, метрики могут выглядеть очень высокими, но при этом не полностью отражать сложность промышленного сценария. Поэтому в отчете отдельно

обсуждается различие между метриками по позитивным запросам и метриками по общей совокупности, включающей вакансии без позитивной разметки.

На уровне всех запросов наблюдается снижение Precision@K до диапазона около 0.28. Это не обязательно означает ошибку модели: показатель отражает состав выборки, где часть вакансий не имеет размеченных позитивных кандидатов. В такой ситуации агрегированная Precision смешивает два эффекта: качество ранжирования там, где релевантные кандидаты есть, и отсутствие позитивных объектов там, где их в разметке нет. Поэтому корректный анализ должен разделять эти случаи.

2.4. Валидационный протокол и ограничения экспериментальной выборки

Тестовый набор включал 521 пару «вакансия–резюме», 423 уникальные вакансии и 510 уникальных резюме. Из общего числа пар 177 были размечены как релевантные, 344 — как нерелевантные. Такая структура полезна для первичной проверки работоспособности пайплайна, но накладывает ограничения на статистическую интерпретацию результатов.

Во-первых, число пар относительно невелико для окончательных выводов о качестве промышленной системы. Во-вторых, распределение кандидатов по вакансиям неоднородно: для части вакансий доступен ограниченный набор кандидатов. В-третьих, наличие вакансий без позитивной разметки влияет на агрегированные показатели. В-четвертых, качество любой supervised или contrastive постановки зависит от надежности исходной разметки: если позитивные и негативные пары сформированы с шумом, модель может обучаться не только на содержательных различиях, но и на артефактах данных.

Поэтому результаты практики следует рассматривать как проверку инженерной состоятельности подхода и как первичную экспериментальную валидацию. Для перехода к более строгой оценке потребуется расширить контрольный набор, обеспечить независимую проверку разметки, добавить срезы по типам вакансий и сравнить модель с несколькими baseline-решениями.

В текущем протоколе особое внимание уделялось воспроизводимости. Экспериментальные скрипты, конфигурации, результаты расчетов и интеграционные отчеты сохранялись как отдельные артефакты. Это позволяет повторить запуск при изменении данных или модели и сравнить результаты не на уровне субъективного впечатления, а по зафиксированным метрикам.

Выводы

1. Задача ранжирования формализуется как оценка релевантности пары «вакансия–кандидат» с последующей сортировкой кандидатов по убыванию score.

2. Косинусная близость эмбедингов является удобной функцией семантического соответствия для bi-encoder архитектуры.
3. Hard filtering должен применяться до семантического скоринга, поскольку обязательные ограничения нельзя полностью делегировать модели.
4. Для оценки качества необходимо использовать несколько метрик: Precision@K, Recall@K, NDCG@K и MRR.
5. Высокие значения метрик на позитивных запросах требуют интерпретации с учетом структуры выборки и наличия вакансий без позитивной разметки.
6. Текущий тестовый набор достаточен для первичной проверки пайплайна, но не заменяет расширенную промышленную валидацию.

3. Архитектура и реализация экспериментального bi-encoder пайплайна

3.1. Подготовка данных и организация воспроизводимого конвейера

Инженерная часть практики была построена вокруг воспроизводимого конвейера в каталоге experiments/scripts. Такой формат выбран сознательно: для исследовательской работы важно не только получить результат, но и иметь возможность повторить его при изменении данных, параметров модели или версии кода. Каждый существенный этап был вынесен в отдельный скрипт, что упрощает отладку и делает структуру эксперимента прозрачной.

Общая логика пайплайна включает следующие шаги:

1. Профилирование исходного набора данных.
2. Очистку и нормализацию текстовых представлений.
3. Формирование train/validation/test разбиений.
4. Подготовка пар «резюме–вакансия» для обучения.
5. Обучение bi-encoder модели.
6. Расчет метрик на валидационной и тестовой выборках.
7. Подготовка ранжирующего модуля для сервисного использования.
8. Интеграционную проверку API и бизнес-сценариев.

Профилирование данных необходимо для выявления проблем до обучения модели. В задачах NLP ошибки на этапе данных часто оказываются важнее, чем выбор конкретной архитектуры. К таким ошибкам относятся пустые тексты, дубли, несогласованные идентификаторы, перекосы в распределении позитивных и негативных пар, а также слишком короткие или слишком длинные документы. В контексте ATS дополнительно важно

контролировать, какие поля попадают в текстовое представление: если в одном случае используются только навыки, а в другом — полный текст резюме, модель будет получать несопоставимые входы.

Нормализация текстов выполнялась с целью привести данные к более устойчивому виду для кодирования. При этом важно не удалить значимую информацию. Например, технологические термины, названия библиотек, языков программирования и фреймворков должны сохраняться, поскольку именно они часто являются смысловыми маркерами соответствия вакансии. Напротив, технические артефакты, мусорные символы и нерелевантные фрагменты могут ухудшать качество эмбедингов.

Отдельное внимание было уделено организации данных для обучения без избыточной нагрузки на оперативную память. В скрипте обучения использовался динамический доступ к парам через StreamingPairDataset и SQLite-индекс. Такой подход позволяет не загружать весь набор данных целиком и делает процедуру более устойчивой при дальнейшем увеличении объема выборки.

3.2. Обучение bi-encoder модели и контроль качества на валидации

Для обучения использовалась модель на базе sentence-transformers и русскоязычного трансформера cointegrated/rubert-tiny2. Выбор компактной модели обусловлен ограничениями практического этапа: необходимо было получить решение, которое можно обучить, проверить и использовать в сервисном контуре без чрезмерных вычислительных затрат.

Обучение выполнялось с использованием ContrastiveLoss. В такой постановке модель получает пары объектов и учится сближать представления релевантных пар, одновременно разводя представления нерелевантных. Для задачи «вакансия–резюме» это естественная постановка: модель должна формировать такое векторное пространство, где подходящее резюме оказывается ближе к вакансии, чем неподходящее.

Валидация осуществлялась с использованием BinaryClassificationEvaluator. Несмотря на то что конечная задача является задачей ранжирования, бинарная валидация пар полезна как промежуточный контроль: она показывает, насколько модель в принципе различает релевантные и нерелевантные пары по значению similarity.

Динамика валидационных метрик по эпохам представлена в таблице 3.1.

Таблица 3.1 — Динамика метрик валидации по эпохам

Эпоха	Steps	Cosine Accuracy	Cosine F1	Cosine Recall
1.0	522	0.9501	0.9218	0.9763
2.0	1044	0.9674	0.9598	0.9853

Эпоха	Steps	Cosine Accuracy	Cosine F1	Cosine Recall
3.0	1566	0.9731	0.9605	0.9870

Рост Cosine Accuracy от 0.9501 до 0.9731 указывает на улучшение способности модели различать пары на валидационной выборке. Значение Cosine F1 также увеличилось от 0.9218 до 0.9605, что говорит о более сбалансированном качестве между точностью и полнотой. Cosine Recall оставался высоким на протяжении всех эпох и достиг 0.9870 к третьей эпохе.

При интерпретации этих значений важно учитывать, что валидационные метрики по парам не полностью эквивалентны метрикам ранжирования. Модель может хорошо отличать релевантные пары от нерелевантных, но итоговый порядок кандидатов зависит от состава кандидатного пула по каждой вакансии. Поэтому после обучения необходим отдельный этап ранжирования и расчета NDCG, Precision@K, Recall@K и MRR.

Сохранение обученной модели выполнялось в каталог `experiments/models/bi_encoder_rubert_tiny2`. Это позволяет отделить экспериментальную стадию от сервисной: ранжирующий модуль использует уже подготовленную модель и не зависит от повторного запуска обучения.

3.3. Ранжирующий модуль и API-контур ATS

Ключевым прикладным компонентом является модуль `ATSRanker`, реализованный в `10_ranking_module.py`. Он связывает модельный слой с прикладной логикой ранжирования. Внутри модуля реализована двухэтапная схема: сначала кандидаты проверяются на соответствие обязательным ограничениям, затем для оставшегося множества выполняется `dense reranking`.

На вход модуль получает описание вакансии и набор кандидатов. Для вакансии формируется текстовое представление и вычисляется эмбединг. Для кандидатов используются предварительно подготовленные или вычисляемые эмбединги резюме. Далее рассчитывается косинусная близость, после чего кандидаты сортируются по `score`. Результат ограничивается параметром `top-K`.

В выходном JSON-ответе фиксируются:

- список кандидатов в порядке убывания релевантности;
- `score` каждого кандидата;
- количество обработанных кандидатов;
- время выполнения запроса;
- версия модели или служебный идентификатор конфигурации;
- дополнительные поля, необходимые для интеграции с ATS.

Сервисный слой реализован в `11_api_server.py` на базе FastAPI. Были предусмотрены эндпоинты для индексации кандидатов, ранжирования по вакансии и точечного скоринга пары «вакансия–кандидат». Такой набор покрывает основные сценарии использования: массовая подготовка кандидатного пула, получение top-K выдачи и диагностическая проверка конкретной пары.

Для наблюдаемости в `middleware` добавляются телеметрические заголовки `X-Processing-Time-Ms` и `X-Model-Version`. Это важно для эксплуатации: пользователь API или интеграционный тест может проверить не только содержимое ответа, но и время обработки, а также версию модели, которая участвовала в расчете. Без такой телеметрии сложно анализировать регрессии и сравнивать разные экспериментальные конфигурации.

API-контур также выполняет роль границы между исследовательским кодом и прикладной системой. Пока модель существует только как `notebook` или локальный скрипт, ее трудно использовать в реальном бизнес-сценарии. После оформления в виде сервиса появляется возможность подключать UI, e2e-тесты и внешние компоненты ATS.

3.4. Управление артефактами практики и связь с этапами задания

Отдельной задачей практики было обеспечить связь между планом работ, кодом, результатами запусков и отчетными материалами. Это важно для производственной практики, поскольку итоговая оценка зависит не только от наличия кода, но и от того, насколько результаты можно проверить и сопоставить с заявленными этапами.

Этапы работы были организованы следующим образом.

Этап 1. Аналитическая постановка задачи.

На этом этапе были систематизированы материалы по теме ATS и интеллектуального ранжирования кандидатов. Рассмотрены особенности рекрутинговых данных, сформулирована задача семантического сопоставления «резюме–вакансия», сопоставлены основные подходы NLP и метрики качества ранжирования. Результатом этапа стали аналитические материалы, обзор источников и структура будущего отчета.

Этап 2. Доработка bi-encoder пайплайна.

В рамках этого этапа были подготовлены скрипты в каталоге `experiments/`, выполнены экспериментальные прогоны, зафиксированы конфигурации и логи, а также получены валидационные метрики. Особое внимание уделялось воспроизводимости: каждый шаг должен быть запускаемым независимо и оставлять проверяемый результат.

Этап 3. Проверка ранжирования в тестовом контуре ATS.

На этом этапе ранжирующий модуль был проверен в условиях, приближенных к прикладному

сценарию. Были подготовлены протоколы проверок, рассчитаны метрики и выполнены интеграционные тесты API.

Этап 4. Подготовка отчетных материалов. Результаты экспериментов и интеграции были оформлены в отчет по практике. Также были сформированы выводы, которые могут быть использованы в пояснительной записке и презентации к последующим этапам магистерской работы.

Этап 5. Сравнение результатов и формирование рекомендаций. На заключительном этапе были сопоставлены результаты ранжирования, сформированы сводные таблицы метрик, описаны риски и предложены направления дальнейшего развития решения.

Такая организация позволила избежать разрыва между кодом и отчетом. Каждый содержательный вывод в отчете связан с определенным артефактом: скриптом, таблицей метрик, файлом результата или интеграционным протоколом.

Выводы

1. Экспериментальный пайплайн построен как воспроизводимая цепочка скриптов, охватывающая подготовку данных, обучение, оценку и интеграционные проверки.
2. Для обучения использована компактная русскоязычная bi-encoder модель, что соответствует ограничениям вычислительных ресурсов и требованиям к сервисной применимости.
3. Валидационные метрики демонстрируют положительную динамику обучения, однако требуют дополнения ранжировочными метриками на уровне вакансий.
4. Модуль ATSRanker реализует практическую схему «обязательная фильтрация + семантическое ранжирование».
5. API-контур на FastAPI обеспечивает возможность интеграции с ATS и поддерживает базовую телеметрию выполнения запросов.
6. Связь между этапами задания, кодом и результатами повышает проверяемость практики и упрощает дальнейшее развитие проекта.

4. Экспериментальная проверка, интеграционное тестирование и анализ результатов

4.1. Результаты ранжирования на тестовой выборке

Экспериментальная проверка ранжирования была выполнена на тестовом наборе, включающем 423 вакансии и 510 резюме. Общее число пар составило 521, из них 177 были

размечены как релевантные, 344 — как нерелевантные. Время расчета метрик на текущем стенде составило 3.87 секунды.

Основные метрики по позитивным запросам представлены в таблице 4.1.

Таблица 4.1 — Макрометрики по позитивным запросам

K	NDCG@K	Precision@K	Recall@K	MRR
1	1.0000	1.0000	0.8471	1.0000
3	1.0000	0.9958	0.9824	1.0000
5	1.0000	0.9958	0.9924	1.0000
10	1.0000	0.9958	0.9981	1.0000

Полученные значения показывают, что на подмножестве запросов с позитивной разметкой модель уверенно поднимает релевантных кандидатов в верхнюю часть выдачи. Значение $MRR = 1.0000$ означает, что первый релевантный кандидат в рассматриваемых позитивных сценариях находится на первой позиции. Высокие значения $NDCG@K$ указывают на корректный порядок релевантных объектов внутри top-K.

Однако интерпретировать эти значения необходимо осторожно. Во-первых, тестовая выборка имеет ограниченный размер. Во-вторых, часть вакансий не имеет позитивных пар, и это влияет на метрики по полной совокупности запросов. В-третьих, при малом числе кандидатов на отдельную вакансию задача ранжирования становится проще, чем в промышленном сценарии с сотнями или тысячами кандидатов. Поэтому эти результаты следует рассматривать как подтверждение работоспособности пайплайна и полезный ориентир для дальнейших сравнений, но не как окончательную оценку качества будущей промышленной системы.

Отдельно следует отметить различие между метриками по позитивным запросам и по всем запросам. При включении вакансий без позитивной разметки $Precision@K$ снижается до диапазона около 0.28. Это ожидаемо: если в кандидатном пуле для части вакансий отсутствуют размеченные релевантные кандидаты, верхние позиции выдачи не могут быть засчитаны как релевантные с точки зрения разметки. Поэтому для корректного анализа необходимо фиксировать не только итоговые значения метрик, но и состав выборки, по которому они рассчитаны.

4.2. Интеграционная проверка API и бизнес-сценариев ATS

Экспериментальная метрика сама по себе не гарантирует пригодность решения для прикладного использования. Модель может показывать хорошие результаты в изолированном скрипте, но не работать в сервисном контуре из-за ошибок контракта API, проблем

сериализации, несовпадения формата входных данных или отсутствия телеметрии. Поэтому в рамках практики был выполнен отдельный интеграционный этап.

Сквозной сценарий интеграционной проверки реализован в `16_e2e_integration_test.py`. Он покрывает типовой цикл использования модуля в ATS:

1. Индексация или подготовка кандидатских профилей.
2. Передача описания вакансии в ранжирующий endpoint.
3. Получение top-K выдачи.
4. Проверка структуры JSON-ответа.
5. Проверка наличия служебных полей и телеметрических заголовков.
6. Диагностический вызов endpoint `/score` для оценки отдельной пары.

Такой тест важен, поскольку он проверяет не только модельную часть, но и договоренность между сервисом и потребителем результата. Для ATS особенно значимо, чтобы ответ был стабильным по структуре: UI и downstream-компоненты должны получать ожидаемые поля независимо от конкретного запроса.

В smoke-отчете интеграции были зафиксированы контрольный и экспериментальный сценарии. Контрольный сценарий имел `compositeScore = 96` при provider `openai`, экспериментальный сценарий — `compositeScore = 93` при provider `biencoder`. Эти значения не следует напрямую трактовать как абсолютное превосходство одного поставщика над другим, поскольку они могут опираться на разные шкалы и механизмы расчета. Их основная ценность — в том, что оба сценария проходят через единый прикладной контур и могут быть сопоставлены по поведению в интерфейсе.

В материалах интеграционной проверки также зафиксированы скриншоты пользовательского процесса: вход в систему, настройки AI, страница вакансии, карточки кандидатов и страница анализа. Это подтверждает, что модуль проверялся не только на уровне функций, но и в контексте реального пользовательского маршрута.

4.3. Анализ устойчивости, рисков и ограничений текущего решения

Несмотря на положительные результаты, текущая версия модуля имеет ряд ограничений, которые необходимо учитывать при дальнейшем развитии.

Размер и структура выборки остаются одним из ключевых ограничений работы. Тестовый набор достаточен для первичной проверки работоспособности подхода, однако он не позволяет полноценно оценить качество на широком спектре вакансий и профилей кандидатов. Для промышленного применения необходима более крупная и разнообразная выборка,

включающая разные профессиональные домены, уровни позиций, типы занятости и профили опыта.

Отдельную сложность создаёт наличие вакансий без позитивной разметки. Такие случаи важны, поскольку позволяют проверить поведение системы в ситуации, когда среди кандидатов может не быть явно релевантных профилей. Однако они усложняют агрегированную интерпретацию метрик. В дальнейшем такие сценарии следует разделять: отдельно анализировать ранжирование при наличии релевантных кандидатов, поведение модели при их отсутствии и качество самого кандидатного пула.

Ещё одно ограничение связано с зависимостью модели от качества текстового представления данных. Bi-encoder получает на вход не исходную базу данных, а заранее сформированный текст вакансии или резюме. Если это представление неполное, зашумлённое или несогласованное, итоговый score будет отражать не только реальное соответствие кандидата вакансии, но и качество подготовки входных данных. Поэтому стандартизация шаблонов представления вакансий и резюме является отдельной важной инженерной задачей.

Также в текущей работе отсутствует полноценное сравнение с baseline-решениями. Для более строгого научного вывода необходимо сопоставить предложенный подход как минимум с лексическими моделями, простой эвристикой по совпадению навыков и, по возможности, cross-encoder моделью на ограниченном top-K. Без такого сравнения нельзя количественно оценить, насколько bi-encoder действительно улучшает качество относительно более простых методов.

Ограничением остаётся и объяснимость результата. Косинусная близость эмбедингов даёт удобный числовой score, но сама по себе не показывает пользователю, какие именно навыки, требования или фрагменты опыта повлияли на позицию кандидата в выдаче. Для прикладного ATS-интерфейса желательно дополнить ранжирование объясняющими признаками: совпадающими навыками, сильными сторонами кандидата, отсутствующими требованиями и кратким текстовым обоснованием соответствия.

После внедрения модели потребуется отдельный мониторинг качества. Даже если система показывает приемлемые результаты на тестовом наборе, в эксплуатации распределение данных может постепенно меняться. Могут появляться новые роли, новые названия технологий, новые форматы резюме и вакансий. Поэтому необходимы мониторинг деградации качества, регулярное обновление контрольных наборов и понятный регламент переобучения модели.

Перечисленные ограничения не отменяют результатов практики, но задают границы их применимости. На текущем этапе был получен работоспособный прототип, который можно проверять, сравнивать и развивать дальше. Следующий этап работы должен быть направлен на

повышение строгости оценки, улучшение устойчивости решения и подготовку системы к более надёжному применению в реальных условиях.

4.4. Рекомендации по дальнейшему развитию модуля

В дальнейшем развитие работы целесообразно вести в нескольких направлениях. Прежде всего необходимо расширить и выровнять датасет: увеличить число размеченных пар и обеспечить более равномерное распределение кандидатов по вакансиям. Отдельное внимание стоит уделить формированию профессиональных срезов, например для backend, frontend, data science, ML engineering, DevOps, аналитики и других ролей. Это позволит оценивать качество модели не только в среднем по всему набору данных, но и внутри конкретных предметных доменов.

Важным направлением является контроль качества разметки. Часть ошибок модели может быть связана не с ограничениями выбранной архитектуры, а с неточностями в исходных позитивных и негативных парах. Поэтому для более надёжной оценки желательно ввести процедуру дополнительной проверки разметки и сформировать небольшой gold-набор, размеченный несколькими экспертами.

Также необходимо сравнить предложенный подход с базовыми моделями. В качестве baseline-решений можно использовать BM25/TF-IDF, простую эвристику по пересечению навыков, а также вариант bi-encoder без дообучения. Такое сравнение позволит более явно показать вклад именно обученного bi-encoder подхода и оценить, насколько он превосходит простые методы ранжирования.

Отдельно можно рассмотреть добавление второго этапа ранжирования для top-K кандидатов. Bi-encoder хорошо подходит для быстрого отбора большого пула резюме, однако для финального уточнения top-20 или top-50 потенциально полезны cross-encoder модели или LLM-based judge. Такой этап имеет смысл внедрять при условии, что требования к latency и стоимости допускают дополнительную вычислительную нагрузку.

Кроме того, перспективным направлением является развитие объяснимости результатов. Для пользователя ATS важно видеть не только итоговый score, но и причину, по которой кандидат оказался в выдаче. Объяснение может включать совпадающие навыки, релевантные проекты, уровень опыта, отсутствующие обязательные требования и краткое резюме соответствия кандидата вакансии.

После интеграции модели в систему необходимо организовать мониторинг в эксплуатации. Следует фиксировать распределение score, время ответа, долю отфильтрованных кандидатов, изменения в top-K и обратную связь пользователей. Эти данные позволяют

своевременно обнаруживать деградацию качества и принимать обоснованные решения о переобучении модели.

Наконец, для устойчивого развития решения требуется улучшение MLOps-процесса. Необходимо версионировать датасеты, модели, конфигурации и результаты экспериментов. Каждый запуск должен быть воспроизводимым: по номеру эксперимента должно быть понятно, какие данные, параметры и версия кода использовались при обучении и оценке модели.

Выводы

1. Экспериментальная проверка подтверждает работоспособность выбранного подхода на текущем тестовом наборе.
2. Высокие значения NDCG и MRR на позитивных запросах показывают, что модель успешно поднимает релевантные пары в верхнюю часть выдачи.
3. Метрики необходимо интерпретировать с учетом структуры выборки, особенно наличия вакансий без позитивной разметки.
4. Интеграционные проверки демонстрируют, что ранжирующий модуль может использоваться через API и поддерживает основной бизнес-сценарий ATS.
5. Основные направления дальнейшего развития связаны с расширением датасета, сравнением с baseline, повышением объяснимости и организацией мониторинга качества.

Заключение

В ходе производственной практики выполнен полный цикл работ по разработке и проверке интеллектуального модуля ранжирования кандидатов для ATS-системы. Работа включала анализ предметной области, формализацию задачи релевантности, выбор архитектуры, реализацию экспериментального пайплайна, обучение bi-encoder модели, расчет метрик, разработку API-контура и интеграционную проверку прикладных сценариев.

Главным результатом практики является не отдельная модель как таковая, а воспроизводимый инженерно-исследовательский контур. Он позволяет подготовить данные, обучить модель, оценить качество, запустить ранжирование через API и проверить результат в сценарии, близком к работе ATS. Такой контур важен для дальнейшей магистерской работы, поскольку создает основу для систематических экспериментов и сравнения различных подходов.

Выбранная архитектура «hard filtering + bi-encoder ranking» показала практическую применимость. Обязательная фильтрация обеспечивает управляемость и соблюдение бизнес-правил, а bi-encoder модель позволяет учитывать семантическое соответствие вакансий и резюме без чрезмерной вычислительной нагрузки. Валидационные и тестовые результаты подтверждают работоспособность решения на текущем наборе данных.

Вместе с тем полученные метрики требуют осторожной интерпретации. Тестовый набор содержит ограниченное число пар и неоднородную структуру запросов, включая вакансии без позитивной разметки. Поэтому результаты следует рассматривать как успешную первичную апробацию, а не как окончательную оценку промышленной точности. Для следующего этапа необходимо расширить выборку, улучшить контроль разметки, провести сравнение с baseline-моделями и сформировать более строгий протокол валидации.

Практическая значимость выполненной работы состоит в том, что разработанный модуль уже имеет признаки интеграционной готовности: он оформлен как сервис, возвращает структурированный результат, поддерживает телеметрию и может быть встроен в пользовательский процесс ATS. Научно-прикладная значимость состоит в том, что работа задает проверяемую постановку задачи семантического ранжирования кандидатов и формирует основу для дальнейшего исследования качества, устойчивости и объяснимости таких моделей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Rosenberger J., Wolfrum L., Weinzierl S., Kraus M., Zschech P. CareerBERT: Matching Resumes to ESCO Jobs in a Shared Embedding Space for Generic Job Recommendations. Expert Systems with Applications, 2025 (arXiv:2503.02056).
2. Fabris A., Rus C., Saldivar J., Gatzoura A., Biega A.J., Castillo C. Does fair ranking lead to fair recruitment outcomes? A study of interventions, interfaces, and interactions. Information Processing & Management, 63 (2026) 104506.
3. IBM Research. PROSPECT: A system for screening candidates for recruitment. CIKM 2010.
4. IBM Research. Matching Resumes to Jobs via Deep Siamese Network. WWW 2018.
5. Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.
6. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv:1908.10084.
7. Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A.N., Kaiser Ł., Polosukhin I. Attention is All You Need. Advances in Neural Information Processing Systems 30 (NIPS/NeurIPS 2017).
8. Liu T.-Y. Learning to Rank for Information Retrieval. Foundations and Trends in Information Retrieval, 3(3), 2009, pp. 225–331.
9. Burges C., Shaked T., Renshaw E., Lazier A., Deeds M., Hamilton N., Hullender G. Learning to Rank using Gradient Descent. Proceedings of ICML 2005. (RankNet).
10. Järvelin K., Kekäläinen J. Cumulated Gain-based Evaluation of IR Techniques. ACM Transactions on Information Systems, 20(4), 2002, pp. 422–446.
11. Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 2009, pp. 333–389.
12. Wang J., Yi X., Guo R. и др. Milvus: A Purpose-Built Vector Data Management System. SIGMOD '21, 2021.
13. Subramanya S.J., Devvrit, Kadekodi R., Krishaswamy R., Simhadri H.V. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. NeurIPS 2019.
14. Chen Q., Zhao B., Wang H. и др. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search. NeurIPS 2021.
15. Gong Z., Zeng Y., Chen L. Accelerating Approximate Nearest Neighbor Search in Hierarchical Graphs: Efficient Level Navigation with Shortcuts. PVLDB, 18(10), 2025, pp. 3518–3530.

16. Singh A., Joachims T. Policy Learning for Fairness in Ranking. NeurIPS 2019.
17. Li J., Xu B., Chen Z. и др. Enhancing Talent Search Ranking with Role-Aware Expert Mixtures and LLM-based Fine-Grained Job Descriptions. EMNLP 2025 Industry Track, pp. 185–194.
18. Senger E., Zhang M., van der Goot R., Plank B. Deep Learning-based Computational Job Market Analysis: A Survey on Skill Extraction and Classification from Job Postings. NLP4HR 2024, pp. 1–15.
19. Snegirev A., Tikhonova M., Maksimova A., Fenogenova A., Abramov A. The Russian-focused embedders’ exploration: ruMTEB benchmark and Russian embedding model design. NAACL 2025, pp. 236–254.
20. Zmitrovich D., Abramov A., Kalmykov A. и др. A Family of Pretrained Transformer Language Models for Russian. arXiv:2309.10931.
21. Gallegos I.O., Rossi R.A., Barrow J. и др. Bias and Fairness in Large Language Models: A Survey. Computational Linguistics, 50(3), 2024, pp. 1097–1179.
22. Wilson K., Caliskan A. Gender, Race, and Intersectional Bias in Resume Screening via Language Model Retrieval. arXiv:2407.20371.
23. Tang F., Zhu R., Yao F., Wang J., Luo L., Li B. Explainable person–job recommendations: challenges, approaches, and comparative analysis. Frontiers in Artificial Intelligence, 8:1660548, 2025.
24. Zhang M., Jensen K.N., Sonniks S.D., Plank B. SKILLSPAN: Hard and Soft Skill Extraction from English Job Postings. NAACL 2022, pp. 4962–4984.

Приложение А. Профилирование набора данных

Профилирование набора данных выполнялось как предварительный этап перед обучением и тестированием модели. Цель профилирования заключалась в том, чтобы оценить состав выборки, выявить возможные ограничения и зафиксировать исходные параметры эксперимента.

В тестовом наборе использовано:

- 521 пара «вакансия–резюме»;
- 423 уникальные вакансии;
- 510 уникальных резюме;
- 177 релевантных пар;
- 344 нерелевантные пары.

Такое распределение показывает, что выборка содержит больше негативных примеров, чем позитивных. Для задач ранжирования это типичная ситуация: в реальном подборе число неподходящих или слабо подходящих кандидатов обычно выше числа релевантных. Однако для обучения и оценки важно контролировать баланс, поскольку чрезмерный перекос может привести к завышенной оценке некоторых метрик или, наоборот, к переоценке консервативного поведения модели.

В ходе профилирования учитывались следующие аспекты:

1. Уникальность идентификаторов. Проверялась корректность связей между вакансиями, резюме и парами. Ошибки идентификаторов могут привести к тому, что модель будет обучаться на неверных соответствиях.
2. Наличие пустых или слишком коротких текстов. Если текст вакансии или резюме практически отсутствует, семантическое кодирование становится неинформативным.
3. Распределение позитивных пар по вакансиям. Для оценки ранжирования важно понимать, сколько релевантных кандидатов приходится на одну вакансию.
4. Дублирование и близкие дубликаты. Повторяющиеся записи могут завышать метрики, если близкие документы попадают одновременно в обучение и тест.
5. Состав текстовых полей. Проверялось, какие части вакансий и резюме включаются в модельное представление.

Отдельное значение имеет наличие вакансий без позитивной разметки. Такие вакансии не являются ошибкой сами по себе: в реальной системе часто возникают ситуации, когда в текущем пуле нет подходящих кандидатов. Однако для расчета метрик ранжирования такие

случаи необходимо обрабатывать отдельно. Если включить их в общий расчет без пояснений, итоговые значения Precision@K будут снижаться, но это снижение будет отражать не только качество модели, а еще и отсутствие релевантных объектов в разметке.

Профилирование также показало необходимость дальнейшего расширения набора данных. Для более строгой оценки желательно сформировать несколько контрольных срезов: по типам ролей, уровням опыта, технологическим стекам и форматам занятости. Это позволит определить, где модель работает стабильно, а где требует дообучения или изменения текстового представления.

Приложение Б. Протокол экспериментальных запусков

Экспериментальные запуски были организованы поэтапно, чтобы каждый результат можно было воспроизвести и связать с конкретным участком пайплайна.

Б.1. Аналитический этап

Срок:

03.05.2026

Содержание работ: систематизация материалов по теме рекрутмента и ATS, постановка задачи семантического сопоставления «резюме–вакансия», сопоставление NLP-подходов и метрик качества ранжирования.

Результат: аналитический обзор, структура отчета, предварительный перечень требований к модулю.

На этом этапе была уточнена прикладная рамка работы. Было зафиксировано, что модуль ранжирования должен учитывать как семантическое сходство, так и обязательные ограничения. Также был выбран основной подход — bi-encoder модель с последующим расчетом косинусной близости.

Б.2. Подготовка и обучение модели

Срок:

08.05.2026

Содержание работ: доработка bi-encoder пайплайна в каталоге experiments/, подготовка пар, настройка обучения, запуск валидации, сохранение модели и результатов.

Результат: код пайплайна, обученная модель, таблица валидационных метрик, конфигурации и логи запусков.

Особое внимание было уделено тому, чтобы обучение не зависело от ручных действий. Скрипты должны позволять повторить эксперимент на той же версии данных и получить сопоставимый результат. Для работы с данными использовался потоковый доступ, что снижает требования к памяти и упрощает масштабирование.

Б.3. Проверка ранжирования

Срок:

10.05.2026

Содержание работ: проверка ранжирования в тестовом контуре ATS, расчет метрик top-K, анализ позитивных запросов и общей совокупности.

Результат: протоколы проверок, сводные таблицы метрик, файл результатов ранжирования.

На этом этапе было важно перейти от оценки отдельных пар к оценке порядка кандидатов по каждой вакансии. Именно ранжировочные метрики отражают конечный пользовательский сценарий: рекрутер видит не бинарное решение по паре, а список кандидатов.

Б.4. Интеграционный этап

Срок:

13.05.2026

Содержание работ: подготовка отчета по практике, формирование выводов для пояснительной записки и презентации, проверка API-контуров.

Результат: отчетные материалы, фрагменты для ПЗ, интеграционные протоколы.

Интеграционная проверка подтвердила, что модельный результат может быть передан во внешний контур ATS. Были проверены структура ответа, телеметрия и основные сценарии вызова.

Б.5. Итоговый анализ

Срок:

20.05.2026

Содержание работ: сравнение результатов ранжирования, обобщение метрик, фиксация рисков и рекомендаций.

Результат: сводные таблицы, перечень ограничений, предложения по дальнейшему развитию.

Ключевым итогом стало формирование воспроизводимого маршрута от подготовки данных до интеграционного теста. Это позволяет использовать практику как основу для дальнейшей магистерской работы и последующих экспериментов.

Приложение В. Материалы интеграционной проверки

Интеграционная проверка была направлена на подтверждение того, что ранжирующий модуль работает не только как отдельный экспериментальный скрипт, но и как компонент прикладного контура ATS.

В рамках проверки использовались следующие материалы:

- experiments/results/reqcore_after_login.png;
- experiments/results/ai_settings_biencoder.png;
- experiments/results/job_page_priya_biencoder.png;
- experiments/results/job_ai_analysis_page.png;
- experiments/results/job_page_fullstack.png;
- experiments/results/control_application_page.png.

Скриншоты фиксируют основные шаги пользовательского маршрута: вход в систему, настройку AI-поставщика, переход к вакансии, отображение кандидатов и анализ результата. Наличие таких материалов важно для практического отчета, поскольку демонстрирует связь между backend-ранжированием и пользовательским интерфейсом.

В ходе интеграционной проверки оценивались следующие аспекты:

1. Корректность запуска сервиса. API должен подниматься без ручных исправлений окружения и быть доступным для тестового клиента.
2. Корректность входного контракта. Сервис должен принимать описание вакансии и список кандидатов в ожидаемом формате.
3. Корректность выходного контракта. Ответ должен содержать top-K выдачу, score и служебные поля.
4. Наличие телеметрии. Заголовки времени обработки и версии модели должны присутствовать в ответе.
5. Поведение endpoint /score. Точечная оценка пары необходима для диагностики и ручной проверки спорных случаев.
6. Согласованность UI и backend-результата. Отображаемая выдача должна соответствовать расчетам ранжирующего модуля.

По итогам проверки можно сделать вывод, что текущий прототип достиг уровня интеграционной готовности, достаточного для дальнейших экспериментов и демонстрационных сценариев.

Приложение Г. Фрагменты конфигурации и рабочих сценариев

В экспериментальном контуре использовались следующие основные компоненты:

- bi-encoder модель на базе cointegrated/rubert-tiny2;
- библиотека sentence-transformers;
- обучение с использованием ContrastiveLoss;
- потоковый датасет StreamingPairDataset;
- SQLite-индекс для доступа к данным;
- ранжирующий модуль ATSRanker;
- FastAPI-сервис;
- e2e-тестирование через TestClient;
- результаты экспериментов в каталоге experiments/results.

Типовой рабочий сценарий можно описать следующим образом.

1. Подготовить исходные данные и проверить их структуру.
2. Сформировать текстовые представления вакансий и резюме.
3. Создать пары для обучения и валидации.
4. Запустить обучение bi-encoder модели.
5. Сохранить модель и конфигурацию эксперимента.
6. Рассчитать метрики на тестовом наборе.
7. Запустить API-сервис.
8. Выполнить интеграционный тест ранжирования.
9. Зафиксировать результаты в отчетных материалах.

Для повторяемости особенно важно фиксировать версии данных и модели. Если изменяется датасет, результаты метрик нельзя напрямую сравнивать с предыдущими запусками без указания версии набора. Аналогично, изменение шаблона текстового представления вакансии или резюме фактически меняет входную постановку задачи, даже если архитектура модели остается прежней.

В дальнейшем конфигурации экспериментов целесообразно вынести в отдельные YAML/JSON-файлы. Это позволит запускать разные варианты модели и данных без изменения кода, а также упростит сравнение результатов.

Приложение Д. Анализ рисков внедрения и качества данных

Разработка интеллектуального модуля ранжирования в ATS связана не только с модельными, но и с организационными рисками. Ниже приведены основные группы рисков, выявленные в ходе практики.

Одним из важных рисков является смещение данных. Если обучающая выборка отражает только узкую часть вакансий или кандидатов, модель может хуже работать на новых доменах. Например, модель, обученная преимущественно на однотипных IT-вакансиях, может некорректно ранжировать кандидатов для смежных ролей. Для снижения этого риска необходимо расширять выборку и отдельно контролировать качество на разных профессиональных срезах.

Также существует риск неполной или шумной разметки. Релевантность пары «вакансия–резюме» не всегда является очевидной и однозначной. Разные эксперты могут по-разному оценивать одного и того же кандидата, особенно если часть требований относится к желательным, а не обязательным. Поэтому разметка должна сопровождаться понятными правилами и, по возможности, проверкой межэкспертного согласия.

Отдельное ограничение связано с риском переоценки качества по небольшой выборке. Высокие значения метрик на ограниченном наборе данных могут создавать ложное впечатление, что решение уже достаточно надёжно. Чтобы избежать этого, модель необходимо проверять на независимых тестовых наборах и сравнивать с простыми baseline-подходами. Если более сложная модель не показывает заметного преимущества над baseline, её внедрение может быть неоправданным.

Следующий риск связан с деградацией качества при изменении рынка и технологий. Технологические стеки, названия ролей и профессиональные термины со временем меняются. Новые библиотеки, фреймворки и требования могут отсутствовать в исходных данных, на которых обучалась модель. Без регулярного обновления контрольных наборов качество системы может постепенно снижаться, поэтому для эксплуатации необходим регламент мониторинга и переобучения.

Важным прикладным риском является непрозрачность результата для пользователя. Если система показывает только итоговый score, пользователь может не понимать, почему конкретный кандидат оказался выше или ниже в выдаче. Для HR-сценариев особенно важно объяснять результат: какие навыки совпали, какие требования не покрыты и какие фрагменты

опыта наиболее релевантны. Поэтому объяснимость должна рассматриваться как часть продукта, а не как необязательное дополнение.

Также необходимо учитывать риск нарушения бизнес-правил. Семантическая модель может поставить кандидата высоко даже в том случае, если он не проходит по обязательному условию вакансии. Поэтому *hard filtering* должен сохраняться как отдельный этап и быть явно проверяемым. Модельный *score* не должен переопределять жёсткие ограничения, заданные в вакансии.

Наконец, существует риск нестабильной эксплуатации. В сервисном контуре важны не только метрики качества, но и *latency*, доступность, контроль версии модели и воспроизводимость результатов. Даже качественная модель не будет полезна, если сервис периодически возвращает разные форматы ответов, не фиксирует используемую версию или не позволяет диагностировать ошибки. Поэтому телеметрия, версионирование и интеграционные тесты должны быть обязательной частью дальнейшего развития системы.

Приложение Е. Методические замечания по воспроизводимости экспериментов

Воспроизводимость является одним из ключевых требований к прикладному ML-исследованию. Для данной работы это особенно важно, поскольку результаты должны быть использованы не только в отчете по практике, но и в дальнейшем развитии магистерской темы.

Минимальный набор воспроизводимого эксперимента должен включать:

1. Версию исходного датасета.
2. Правила формирования текстовых представлений вакансий и резюме.
3. Разбиение на train/validation/test.
4. Версию модели и библиотеки обучения.
5. Гиперпараметры обучения.
6. Random seed и параметры окружения.
7. Путь к сохраненной модели.
8. Файл рассчитанных метрик.
9. Логи запуска.
10. Описание отличий от предыдущего эксперимента.

В текущем проекте значительная часть этих требований уже реализована через структуру experiments/, сохранение результатов и интеграционные отчеты. Следующим шагом может стать введение единого реестра экспериментов, в котором каждый запуск получает идентификатор и краткое описание.

Пример записи в таком реестре может включать:

- experiment_id;
- дату запуска;
- версию данных;
- базовую модель;
- loss-функцию;
- число эпох;
- размер batch;
- итоговые validation-метрики;
- итоговые ranking-метрики;
- ссылку на модельный артефакт;
- комментарий о назначении запуска.

Такой подход позволит избежать ситуации, когда результаты существуют только в виде разрозненных файлов. Для научной и инженерной работы важно, чтобы любой вывод можно было проследить до конкретного запуска и конкретной версии данных.

Отдельно следует фиксировать изменения текстового представления. Для bi-encoder моделей это критично: даже небольшое изменение шаблона, например добавление раздела с навыками или удаление описания проекта, может заметно изменить эмбединги и итоговый порядок кандидатов. Поэтому шаблон представления должен версионироваться так же, как модель и датасет.

В перспективе целесообразно дополнить экспериментальный контур автоматическими регрессионными проверками. При изменении кода или модели система должна автоматически рассчитывать метрики на контрольном наборе и сигнализировать, если качество ухудшается. Это позволит использовать модуль не только как исследовательский прототип, но и как поддерживаемый инженерный компонент.